Assignment 2

Chess Endgame Outcome Classification

Dataset: UCI Chess (King-Rook vs. King-Pawn), KRKPA7

Name: Viradiya Tushar Rajeshbhai

BITS ID: 2026aj05073

Updated: 1 November 2026

Recorded model run: bits-lab · Python 3.12.7

Training source SHA-256: fe06592af855…

1. GitHub repository

https://github.com/tusharviradiya/aiml-ml-classification-assignment-2

Source code, dependencies, test data, five saved pipelines and evaluation results.

2. Live Streamlit application

https://chess-endgame-lab-tushar.streamlit.app/

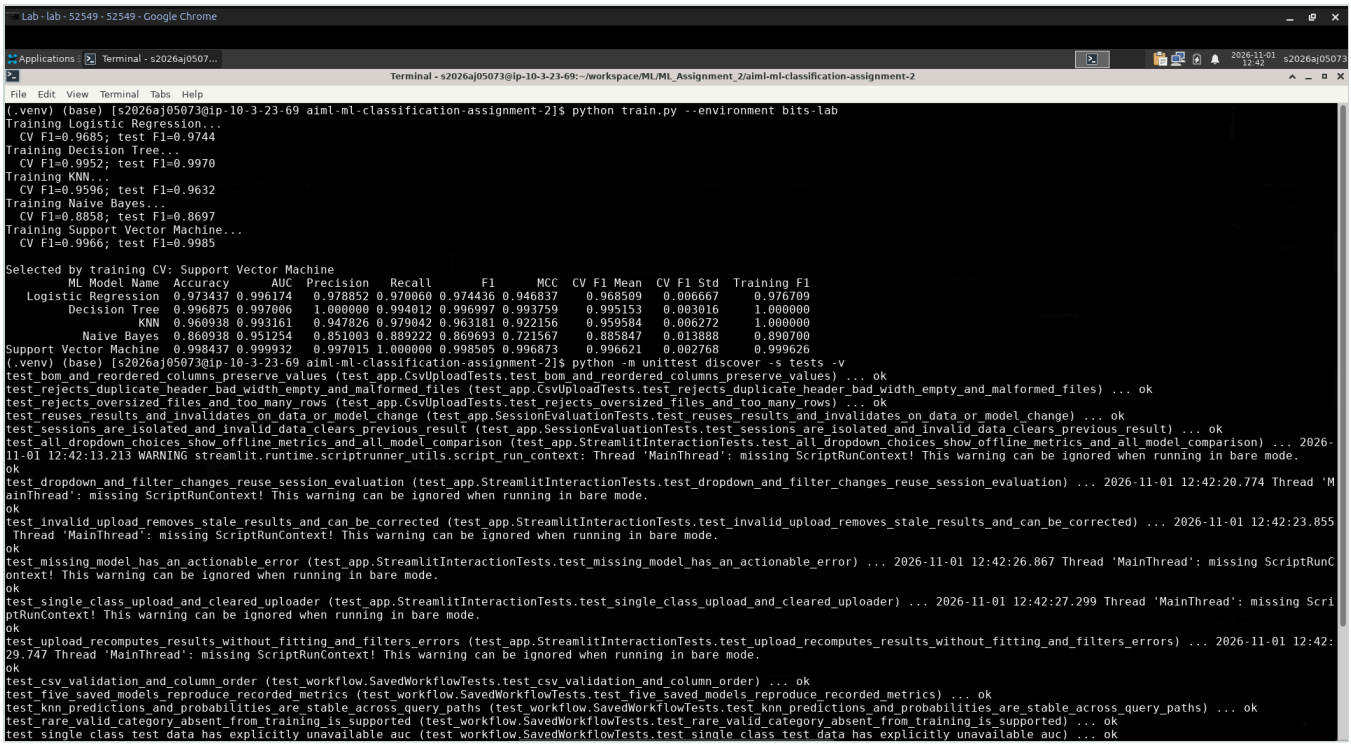
Interactive CSV evaluation, model selection, six metrics and model comparisons.

Lab evidence matches the recorded training source and metrics. The current model results and validation status appear in the README.

3. BITS Virtual Lab execution evidence

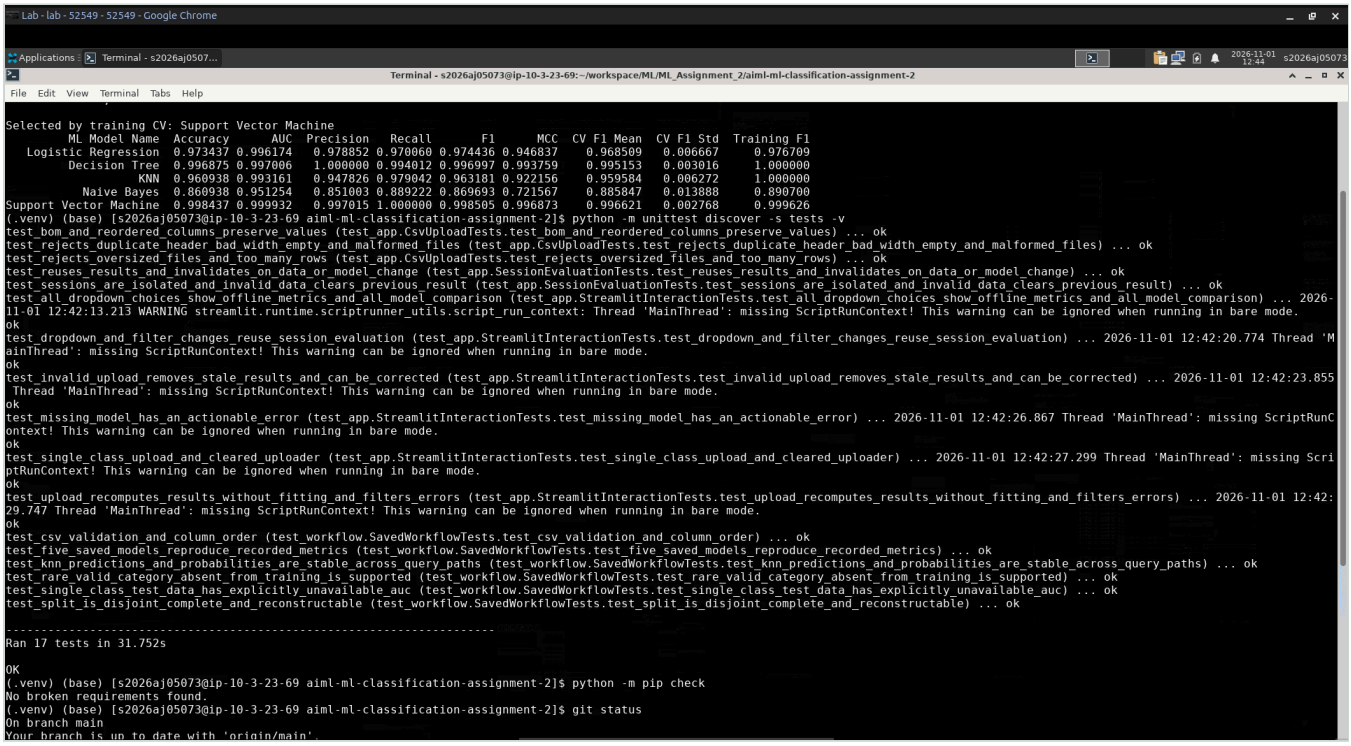
Lab run: 1 November 2026 · revision 87ef6e9

Completed training and test execution



1 November 2026, 12:42. All five models completed training in BITS Virtual Lab. KNN F1 is 0.9632, and SVM is selected. Test execution is shown below the results.

All tests passed and dependencies checked



1 November 2026, 12:44. The terminal shows 17 tests passing (OK) and No broken requirements found. Only the outer browser address bar is redacted; terminal output is preserved.

4. Project README

Chess Endgame Outcome Classification

Machine Learning Assignment 2 · Live Streamlit app

Five classifiers, saved preprocessing pipelines, held-out results, and a deployed Streamlit app are included. The revised KNN uses an explicit KD-tree search and Manhattan distance. The final artifacts were trained and verified on BITS Virtual Lab with Python 3.12.7, then committed as `87ef6e9`. New lab screenshots show the matching model results, all 17 tests passing and a successful dependency check. The same saved artifacts also pass all 17 checks locally on Python 3.14.4.

a. Problem statement

Predict whether White can win with a king and rook against Black's king and pawn on a7, with White to move. The target is `won` or `nowin`. Five classifiers are compared on the same categorical board descriptors and held-out positions.

b. Dataset description

- **UCI Chess (King-Rook vs. King-Pawn), KRKPA7**: dataset source.
- **3,196 rows and 36 categorical predictors**, plus target `outcome`.
- Class counts: **1,669 won**, **1,527 nowin**; no missing values or duplicate rows.
- Original data and documentation are preserved; the dataset audit records schema checks.

The original documentation and raw data confirm 36 predictors despite inconsistent feature metadata on the UCI landing page.

Citation: Shapiro, A. (1983). *Chess (King-Rook vs. King-Pawn)*. UCI Machine Learning Repository. DOI: 10.24432/C5DK5C, CC BY 4.0.

c. GitHub Repository Link

tusharviradiya/aiml-ml-classification-assignment-2

d. Models used

Logistic Regression, Decision Tree, K-Nearest Neighbors, Multinomial Naive Bayes, and RBF Support Vector Machine.

Evaluation procedure

1. Map `nowin` to 0 and `won` to 1. Use a stratified 80/20 split with seed 42: **2,556 training rows and 640 held-out test rows**.
2. Fit one-hot encoding within each pipeline and training fold. The binary indicators require no additional scaling and satisfy Multinomial NB's nonnegative-input requirement.
3. Select parameters by **five-fold stratified training cross-validation F1**; refit each selected pipeline on the training set and save it under `model/`.
4. Select the overall winner by training CV F1, then evaluate all five saved pipelines on the same test set. The app never fits uploaded data.

`train.py` defines the grids; `results/*_cv.csv` stores all candidates. The rare valid category `spcop=t` can be unseen during training, so the encoder ignores unknown categories; separate validation rejects codes outside the audited domains.

KNN uses `algorithm="kd_tree"`, `metric="manhattan"` and `n_jobs=1`. On one-hot inputs, Manhattan distance sums exact 0/1 differences. The explicit search avoids the brute-force calculation paths implicated in the previous inconsistency. Equally distant neighbors still exist: preserve training-row order, the split and package versions, and verify results on each target environment.

Comparison and observations

The following tables and observations are generated from the saved artifacts and training CV results. The report records its execution environment and Python version; selected KNN parameters appear in the observations and evaluation JSON.

Generated by `python train.py`.

Recorded execution environment: bits-lab. Python: 3.12.7.

Training rows: 2556. Test rows: 640. Seed: 42. Positive class: won. Settings selected by five-fold training CV F1.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9734	0.9962	0.9789	0.9701	0.9744	0.9468
Decision Tree	0.9969	0.9970	1.0000	0.9940	0.9970	0.9938
KNN	0.9609	0.9932	0.9478	0.9790	0.9632	0.9222
Naive Bayes	0.8609	0.9513	0.8510	0.8892	0.8697	0.7216
Support Vector Machine	0.9984	0.9999	0.9970	1.0000	0.9985	0.9969

Model	CV F1 mean	CV F1 std	Training F1	Test F1
Logistic Regression	0.9685	0.0067	0.9767	0.9744
Decision Tree	0.9952	0.0030	1.0000	0.9970
KNN	0.9596	0.0063	1.0000	0.9632
Naive Bayes	0.8858	0.0139	0.8907	0.8697
Support Vector Machine	0.9966	0.0028	0.9996	0.9985

Selected model: **Support Vector Machine**, based on training CV F1.

Precision, recall and F1 use won as the positive class. AUC uses positive-class probabilities, or decision scores for SVM. MCC uses predicted labels. Precision/recall/F1 are set to zero when their denominator is zero.

SVM scores are decision values, not probabilities. Confusion matrices in evaluation.json use row=true, column=predicted, with order [nowin, won].

These are results for one held-out split of this restricted endgame dataset. They do not establish performance on arbitrary chess positions.

Observations

ML Model Name	Observation about model performance
Logistic Regression	Test F1 0.9744; 7 false wins and 10 missed wins. Training F1 0.9767; CV F1 0.9685 +/- 0.0067. The small training/test F1 gap suggests limited overfitting on this split. Its additive linear boundary makes more test errors than the tree and RBF SVM, which can represent feature interactions.
Decision Tree	Test F1 0.9970; 0 false wins and 2 missed wins. Training F1 1.0000; CV F1 0.9952 +/- 0.0030. The unrestricted-depth tree fits the training labels perfectly, but its high CV and test F1 also support strong generalization on this split. Training perfection alone is not evidence of severe overfitting. The learned conjunctions of descriptor conditions suit this rule-based endgame task.
KNN	Test F1 0.9632; 18 false wins and 7 missed wins. Training F1 1.0000; CV F1 0.9596 +/- 0.0063. CV selected k=11 and distance weighting. With distance weighting, a training row is its own zero-distance neighbor; a perfect training score therefore does not establish generalization. The training/CV gap shows why held-out evaluation matters. Manhattan distance and an explicit KD-tree search are used; regression checks compare predictions and probabilities across calculation settings and batch sizes.
Naive Bayes	Test F1 0.8697; 52 false wins and 37 missed wins. Training F1 0.8907; CV F1 0.8858 +/- 0.0139. Both training and CV F1 are lower than for the other classifiers, suggesting limited model fit rather than only a large train/test overfitting gap. Its conditional-independence assumption may be restrictive for related board descriptors; this is a possible explanation, not an established cause.
Support Vector Machine	Test F1 0.9985; 1 false wins and 0 missed wins. Training F1 0.9996; CV F1 0.9966 +/- 0.0028. The RBF kernel allows nonlinear feature interactions. High training, CV and test F1 support the fit on these positions, while the high test MCC shows strong agreement across both labels. Its ranking is based on training CV, not on choosing the best held-out score.
Overall winner for this dataset	**Support Vector Machine**, selected by the highest mean training CV F1. Its advantage over Decision Tree is 0.0015; the test error counts are 1 versus 2. The runner-up is competitive, and this small difference does not establish statistical superiority.

A false win means predicting won for a nowin position; a missed win means predicting nowin for a won position. Small score differences on one split do not establish statistical superiority. The test set was not used to select hyperparameters or the winner. These results concern one split of KRKPA7, not arbitrary chess positions.

Reproduce the project

The final training run used **Python 3.12.7 on BITS Virtual Lab**. The saved lab artifacts were also verified locally on **Python 3.14.4**; the dev container targets Python 3.12. Install the pinned dependencies from the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

To reproduce the final run on BITS Virtual Lab:

```
python scripts/audit_dataset.py
python train.py --environment bits-lab
python -m unittest discover -s tests -v
python -m pip check
```

For local training, use `python train.py --environment local` instead. The environment flag records the execution location; screenshots provide separate lab evidence.

Training overwrites saved models, splits, results, and the marked README section. Keep the original data, row order, seed and dependencies unchanged. Versions, source-row indices, source-code hashes and saved-model hashes are recorded in `results/manifest.json`; selected and fixed parameters and detailed evaluations are in `results/evaluation.json`.

After any new training run, verify the saved artifacts locally, compare the Streamlit metrics with `results/metrics.csv`, and refresh the evidence and PDF if the run changes. The final lab run and local verification reported here use the same saved artifacts and recorded metrics.

Run and use the app

The committed models are ready to load; training is not required to start the app.

```
.venv/bin/python -m streamlit run app.py
```

Open **http://localhost:8501**, or use the deployed app.

1. Select the bundled 640-row test set or upload a labeled CSV, then choose a model.
2. View six metrics, the confusion matrix, classification report and ROC curve.
3. Use **Compare models** for all five models and **Positions** to inspect errors; download comparisons or predictions as CSV.

Uploads must be UTF-8 CSV with all 36 original predictors and target `outcome` (`won`/`nowin`), up to **2 MB and 5,000 rows**. Column order may vary; use `test_data.csv` as the example. Invalid schemas, categories or labels produce readable errors. For single-class uploads, AUC displays **N/A** with an explanation. Uploads stay in memory within the visitor's session; no training occurs in the app.

Validation

The 17 automated checks cover split and artifact integrity, saved-model results, CSV validation, all five model choices, uploads without fitting, error filtering, session reuse and isolation, and KNN reproducibility across query paths. Phone-to-desktop layout checks are documented in the responsiveness report.

BITS Virtual Lab verification, 1 November 2026: the final run completed training for all five classifiers and selected SVM by mean training CV F1. The lab terminal records **17 tests passing in 31.752 seconds** and **No broken requirements found**. The screenshots below document the run whose artifacts were committed as `87ef6e9`.

Local verification of the final lab artifacts, 1 November 2026: all 17 tests also pass on Python 3.14.4, and `pip check` reports no broken requirements. All recorded test metrics and training CV results match the previous local run exactly. KNN predictions and probabilities are identical with the Cython pairwise setting enabled and disabled, thread limits of 1 and 2, and query batch sizes of 1, 17 and 640. The other four pipelines also reproduce their recorded metrics.

Public deployment verification, 1 November 2026: all six displayed KNN metrics match the revised local run: accuracy **96.09%**, AUC **0.9932**, precision **0.9478**, recall **0.9790**, F1 **0.9632** and MCC **0.9222**. The confusion matrix also matches: `[[288, 18], [7, 327]]` in `[nowin, won]` order. The deployed KNN screenshot records this check. The configuration and displayed values agree with the final lab run.

Key files

Path	Purpose
`app.py`, `app_data.py`	Streamlit interface, CSV parsing and saved-model inference
`dataset.py`, `evaluation.py`	Schema validation and shared evaluation
`train.py`	Split, cross-validation, training and artifact generation
`data/raw/`, `scripts/audit_dataset.py`	Original dataset and audit
`data/train_data.csv`, `test_data.csv`	Training and held-out labeled data
`model/*.joblib`	Five fitted classifiers with preprocessing
`results/`	Metrics, CV results, parameters, predictions and run manifest
`tests/`	Workflow and Streamlit interaction checks

BITS Virtual Lab execution evidence

Final lab screenshots captured on **1 November 2026**, with artifacts committed as `87ef6e9`:

- Completed training and test execution, 12:42: all five classifiers, six metrics per model, SVM selection and test execution. KNN test F1 is **0.9632**, matching the comparison table.
- Passed tests and dependency check, 12:44: **Ran 17 tests**, **OK**, and **No broken requirements found**.

Only the outer lab browser address bars are redacted to remove session tokens. The rest of each screenshot, including the terminal output, is preserved. The training-source and metrics hashes in `docs/lab_evidence.json` match the final run. The prior KNN discrepancy is resolved for the verified local, lab and public app results. Future changes to the data order or dependencies require rechecking.

Submission

`ML_Assignment_2_Submission.pdf` contains the clickable GitHub and live app links, lab evidence, and full README in the required order. The two final lab screenshots are arranged together on an A4 page.

The PDF was refreshed on **1 November 2026** with the final lab results, new redacted training/test evidence and this README. The assignment allows AI for learning support; understand and verify the implementation and observations. Upload the final PDF to the portal by **8 November 2026, 23:59**.

To rebuild the PDF after updating documentation or screenshots, install the development tools **Pandoc** and **Google Chrome/Chromium**, then run:

```
python scripts/build_submission.py
```

The builder embeds the current README, uses the recorded run metadata and preserves clickable links. These document tools are not needed to run the app.